

Reviewer Pack — Minimal Noncommutative L_p Norm of Matrix Entanglement State...

Entry f26beae92900 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-23 17:57:59 UTC

Minimal Noncommutative L_p Norm of Matrix Entanglement States vs Disjointness Communication Complexity

Entry ID: f26beae92900 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-23 17:57:59 UTC

1. Conjecture

Field A (mathematical branch): Noncommutative L^p Geometry **Field B** (complexity object): Communication Complexity (Disjointness)

Statement:

[‘For any $n \times n$ matrix M representing a quantum entanglement state, the minimal non-commutative L_p norm of M , defined as $\min_{1 \leq i \leq j \leq n} \|M\|_{p^2}^{i+j-2}$, satisfies: $\tau(\text{DISJ}_n) = \Omega(n)$, where τ is the noncommutative L_p norm.’, ‘In other words, for any instance of Disjointness with n elements, there exists a matrix M representing an entanglement state such that its noncommutative L_p norm satisfies the relation $\tau(M) \geq cn$ for some constant $c > 0$ and all sufficiently large n .’, ‘Moreover, this lower bound holds for all p in $[1, \infty]$.’]

Rationale (proposer’s reasoning):

[‘Noncommutative L^p geometry provides a framework to study quantum entanglement, which is closely related to communication complexity. By exploring the noncommutative L_p norm of matrix entanglement states, this conjecture proposes a new direction for understanding the fundamental limits in communication complexity.’, ‘The potential link between noncommutative L^p norms and communication complexity of Disjointness could reveal deeper connections between quantum mechanics and computational complexity theory.’, ‘This approach is novel as it bridges noncommutative geometry with communication complexity, which has not been extensively explored in the context of complexity theory.’]

Taxonomy category: COMM_DISJ (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): e5f22a10893472f4

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all $n \leq 40$ and $p \in [1, \infty]$, at least 25 out of 30 random seeds yield a noncommutative L_p norm of the matrix $M \geq cn$ with an error margin of $\pm 10\%$. The conjecture is falsified if any seed produces a norm $< cn$ or $> cn + 10\%$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "noncommutative L^p norm" AND "matrix entanglement states" - "communication complexity" AND "disjointness problem" AND "quantum entanglement" - " L_p norm" IN noncommutative geometry AND communication complexity lower bound

Top relevant hits considered: - [<http://arxiv.org/abs/1209.3595v2>] Noncommutative complex differential geometry - [<http://arxiv.org/abs/1612.06688v2>] The Ricci Curvature in Noncommutative Geometry - [<http://arxiv.org/abs/math/0610043v5>] Lecture Notes on Noncommutative Algebraic Geometry and Noncommutative Tori

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A):
        n = len(A)
        for i in range(n):
            max_row = max(range(i, n), key=lambda r: abs(A[r][i]))
            A[i], A[max_row] = A[max_row], A[i]
            pivot = A[i][i]
            if pivot == 0:
                continue
            for j in range(n):
                A[i][j] /= pivot
            for k in range(n):
                if k != i:
                    factor = A[k][i]
                    for j in range(n):
```

```

        A[k][j] -= factor * A[i][j]

    return A

def matrix_multiplication(A, B):
    n = len(A)
    C = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C

def noncommutative_Lp_norm(M, p):
    n = len(M)
    sum_val = 0
    for i in range(n):
        for j in range(i + 1, n):
            norm_ij = abs(sum(M[i][k] * M[j][k] for k in range(n)))
            if norm_ij != 0:
                sum_val += norm_ij ** (p / (i + j - 2))
    return sum_val

def generate_entanglement_matrix(n):
    # Simplified entanglement matrix generation
    M = [[random.choice([-1, 1]) for _ in range(n)] for _ in range(n)]
    return gaussian_elimination(M)

n = random.randint(5, 40)
M = generate_entanglement_matrix(n)
p_values = [1] + list(range(2, 10)) + [math.inf]
c = 1.0 / n

results = []
for p in p_values:
    norm = noncommutative_Lp_norm(M, p)
    if norm < c * n:
        return {
            "metric_name": "Noncommutative L_p Norm",
            "metric_value": norm,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": f"Norm {norm} is less than cn = {c * n}"
        }
    results.append(norm)

return {
    "metric_name": "Noncommutative L_p Norm",
    "metric_value": sum(results) / len(results),
    "instances_tested": len(p_values),
    "conjecture_holds": True,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

```

```

results = []

for seed in seeds:
    trial_result = run_trial(seed)
    print(f"TRIAL: {trial_result}")
    results.append(trial_result["metric_value"])

mean = sum(results) / len(results)
std = math.sqrt(sum((x - mean) ** 2 for x in results) / len(results))
support_fraction = sum(1 for r in results if r >= c * n) / len(results)

if support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean} std={std} support_fraction={support_fraction}")
elif any(r < c * n for r in results):
    first_failing_seed = next(i + 1 for i, r in enumerate(results) if r < c * n)
    print(f"RESULT: FALSIFIED counterexample='Norm less than cn' first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE reason=insufficient_evidence")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

TRIAL: {'metric_name': 'Noncommutative L_p Norm', 'metric_value': 0, 'instances_tested': 1, 'conjecture': 'True'}
TRIAL: {'metric_name': 'Noncommutative L_p Norm', 'metric_value': 0, 'instances_tested': 1, 'conjecture': 'True'}
TRIAL: {'metric_name': 'Noncommutative L_p Norm', 'metric_value': 0, 'instances_tested': 1, 'conjecture': 'True'}
TRIAL: {'metric_name': 'Noncommutative L_p Norm', 'metric_value': 0, 'instances_tested': 1, 'conjecture': 'True'}
TRIAL: {'metric_name': 'Noncommutative L_p Norm', 'metric_value': 0, 'instances_tested': 1, 'conjecture': 'True'}

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fla02d96.py", line 94, in <module>
    trial_result = run_trial(seed)
                   ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fla02d96.py", line 69, in run_trial
    norm = noncommutative_Lp_norm(M, p)
           ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fla02d96.py", line 54, in noncommutative_Lp_norm
    sum_val += norm_ij ** (p / (i + j - 2))
               ~~~~~^~~~~~
ZeroDivisionError: division by zero

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data that could confirm or falsify the conjecture.
| next: Investigate and fix the crash in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13480
2	propose	ollama_remote	glm4:latest	0	0	11465
3	preregistration	ollama_remote	glm4:latest	0	0	6023
4	novelty	ollama_remote	glm4:latest	0	0	4807
5	novelty	ollama_remote	glm4:latest	0	0	6253
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	42032
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7720
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13077
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10904
10	judge	ollama_remote	glm4:latest	0	0	11258

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 127019 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in *research/audit/f26beae92900.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/f26beae92900.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/f26beae92900.tar.gz* (if generated)